Лабораторная работа №7. Коллективные операции в MPI

Выполнил: Москаленко Дмитрий, группа 11-401

1. Что делает программа

Программа вычисляет сумму массива $a_i = i + 1$ ($i = 0, \ldots, N - 1$) и сравнивает её с $N(N + 1)/2$. Процесс 0 получает N из командной строки, формирует массив и раздаёт его части; каждый процесс считает сумму своей части (long long), затем общая сумма получается тремя способами. Если N не делится на P, массив раздаётся через MPI_Scatterv: первые N mod (остаток) P процессов получают на один элемент больше (доп. задание).

Операция	Назначение в программе	Где результат
MPI_Bcast	передать N от процесса 0 всем	у всех процессов
MPI_Scatter(v)	раздать части массива: по N/P элементов (Scatterv - части разного размера)	каждый процесс получает свою часть
MPI_Gather	собрать локальные суммы в массив, сложить циклом на процессе 0	массив сумм у процесса 0
MPI_Reduce	сложить локальные суммы операцией MPI_SUM	одна сумма у процесса 0
MPI_Allreduce	то же сложение; каждый процесс считает свою долю от общей суммы	одна сумма у всех процессов

2. Компиляция и запуск

MS-MPI (MSYS2), GCC 16.2.0, процессор Intel Core i5-12450H (8 ядер, 12 логических процессоров).

```
mpicc -O2 -Wall -Wextra mpi_array_sum.c -o mpi_array_sum.exe
mpiexec -n 4 ./mpi_array_sum.exe 1000000
```

Запуск выводит суммы с проверкой, время вычисления (от формирования массива до MPI_Allreduce) и время одной операции Gather + сумма и Reduce (1000 повторов). В таблицах - среднее из 3 запусков.

3. Результаты

Таблица 1. Проверка корректности (* - N не делится на P, MPI_Scatterv)

P	N	Ожидаемая	Gather	Reduce	Allreduce	Верно?
2	100	5050	5050	5050	5050	да
4	100	5050	5050	5050	5050	да
4	10000	50005000	50005000	50005000	50005000	да
8	1000000	500000500000	500000500000	500000500000	500000500000	да
4*	10	55	55	55	55	да
3*	1000001	500001500001	500001500001	500001500001	500001500001	да

Таблица 2. Влияние размера массива (P = 4)

N	10^4	10^5	10^6	10^7
Время, с	0,000158	0,000330	0,001858	0,018766
Корректно?	да	да	да	да

Таблица 3. Влияние числа процессов (N = 10^6, базовая конфигурация P = 1)

N	P	Время, с	Ускорение S	Эффективность E
10^6	1	0,001948	1,00	1,00
10^6	2	0,001910	1,02	0,51
10^6	4	0,001858	1,05	0,26
10^6	8	0,002178	0,89	0,11

Таблица 4. Gather + сумма и Reduce (время одной операции агрегирования)

P	Способ	Повторений	Время операции, с	Корректно?
2	Gather + сумма	1000 × 3	$1{,}91 \cdot 10^{-7}$	да
2	Reduce	1000 × 3	$2{,}23 \cdot 10^{-7}$	да
4	Gather + сумма	1000 × 12	$2{,}00 \cdot 10^{-6}$	да
4	Reduce	1000 × 12	$1{,}80 \cdot 10^{-6}$	да
8	Gather + сумма	1000 × 3	$3{,}47 \cdot 10^{-6}$	да
8	Reduce	1000 × 3	$5{,}78 \cdot 10^{-6}$	да

4. Вывод

Все три способа дают правильную сумму, но решают разные задачи: MPI_Gather только собирает значения в массив на процессе 0, а складывать их приходится самому; MPI_Reduce сразу объединяет значения операцией MPI_SUM и отдаёт результат только процессу 0; MPI_Allreduce отдаёт результат всем процессам, поэтому он нужен, когда общий итог требуется каждому процессу для дальнейших вычислений: например, для нормировки своих данных или вычисления отклонений от общего среднего. В программе это показано на простом примере: каждый процесс делит свою локальную сумму на общую и получает свою долю (при N = 100 у процесса 3 - 2200 из 5050, то есть 43,56 %).

Для суммирования Reduce удобнее Gather: это один вызов без вспомогательного массива и цикла, а по времени способы близки.